

Analysedocument V3 - Frontend AI-chatbot Sociaal Knooppunt

Auteur: Sahel Nawabi

1. Inleiding

Dit document beschrijft de analyse die ten grondslag ligt aan het ontwerp en de bouw van de frontend van de AI-chatbot voor Renders Education (Carla Renders). De chatbot, met de werknaam **Padvinder**, helpt inwoners van gemeente Boxtel (15–99 jaar) bij het vinden van passende leefstijl- en welzijnsactiviteiten in de wijken **Boxtel**, **Esch**, **Lennisheuvel** en **Liempde**. De frontend wordt gebouwd met **React** en **Vite** en is uiteindelijk bereikbaar via www.sociaal-knooppunt.nl.

Deze V3-versie is een herziening van de eerste analyse (Sahel Nawabi, 2026a) op basis van nieuwe interne documenten uit het AI-team: de synthetische activiteiten-dataset (Tran, 2026a), de actieve system-prompt-versie V2.1 met scoringsregels (Tran, 2026b), de schrijfrichtlijn voor activiteitenbeschrijvingen (Tran, 2026c), de AVG-checklist op PoC-niveau (Tran, 2026d), de prompt-changelog (Tran, 2026e) en ADR-001 over de slidermeting (Tran, 2026f). Deze documenten leggen het feitelijke datacontract en de gedragsregels vast waarop de frontend moet aansluiten.

De analyse is opgezet volgens het **DOT Framework** (Development Oriented Triangulation) van HBO-i. Dit framework onderscheidt vijf onderzoeksstrategieën, Library, Field, Lab, Showroom en Workshop, en helpt om ontwerpkeuzes onderbouwd, navolgbaar en toetsbaar te maken (HBO-i).

2. Doelstelling en onderzoeksvragen

2.1 Doelstelling

Een onderbouwde aanpak ontwikkelen voor het ontwerpen en bouwen van een toegankelijke, gebruiksvriendelijke en privacy-bewuste frontend voor de AI-chatbot, die het gespreksmodel **Het Leefstijlroer** ondersteunt, aansluit op de doelgroep en het happy-flow-proces, en correct samenwerkt met het backend-contract (slider-scores, RAG-CONTEXT, opt-in account- en contactaanvraagstromen) (Wieringa, 2026a; Wieringa, 2026b; Tran, 2026b; Tran, 2026d).

2.2 Hoofdvraag

Hoe ontwerp en bouw ik een toegankelijke, gebruiksvriendelijke en op B1-taalniveau ontsloten React/Vite-frontend voor de AI-chatbot Sociaal Knooppunt, zodanig dat de happy flow van het Leefstijlroer correct wordt afgehandeld, het datacontract van het AI-team consistent gevolgd wordt en de privacy van de gebruiker geborgd blijft?

2.3 Deelvragen

1. Welke frontend-architectuur en componentstructuur past het beste bij een conversationele interface met sliders, keuzeknoppen en vrije tekst?
2. Op welke manier kan React + Vite ingezet worden om snelle iteratie en een goede ontwikkelervaring te combineren met een lichte, lokaal draaiende productieoplossing?
3. Welke toegankelijkheidseisen (WCAG 2.2 AA, B1-taalniveau) zijn relevant voor een breed publiek van 15 tot 99 jaar en hoe verwerk ik die in het ontwerp?
4. Welke conversational-UI-patronen zijn passend voor een gestructureerd, deels script-gestuurd, deels vrij-gespreksmodel zoals het Leefstijlroer?
5. Hoe ondersteunt de frontend privacy by design en de drie instroompaden ("precies / een beetje / nog niet")?
6. Hoe sluit de frontend-state aan op het datacontract van het AI-team (sliderscores, USER SCORES, RAG-CONTEXT, opt-in account en contactaanvraag)?
7. Hoe maakt de frontend de **matchingvariabelen transparant** zoals geëist in de AVG-checklist en door Carla Renders?

3. DOT Framework: keuze van strategieën

Voor dit deelproject worden alle vijf DOT-strategieën ingezet:

Strategie	Inzet binnen dit project
Library	Literatuuronderzoek naar React/Vite, WCAG 2.2, B1-taalniveau, conversational UI-patronen en het Leefstijlroer.
Field	Doelgroepanalyse en stakeholdergesprekken met Carla Renders; interpretatie van de happy flow, de schrijfrichtlijn en de AVG-checklist als documenten uit het werkveld van het projectteam.
Workshop	Prototyping en componentontwerp; iteratief opstellen van wireframes, een componentbibliotheek en het datacontract met het AI-team.
Lab	Usability- en toegankelijkheidstests (handmatig en met vitest-axe), unit tests op slider- en reducerlogica, schermlezer tests, prompt-injectietests op de UI-invoer.
Showroom	Expertreview van wireframes, componentstructuur en toegankelijkheid door Fontys-docenten/experts (frontend, UX, accessibility); referentiestudie van bestaande chatinterfaces; demo aan stakeholder Carla Renders.

4. Library-onderzoek

4.1 Het Leefstijlroer als inhoudelijk kader

Het Leefstijlroer van Vereniging Arts en Leefstijl is een gespreksinstrument dat leefstijl benadert via zes pijlers: **bewegen, voeding, ontspanning, slaap, verbinding en middelen** (Vereniging Arts en Leefstijl, 2025). Voor de frontend betekent dit dat de UI per pijler eenzelfde structuur ondersteunt: een sliderbeoordeling, eventuele motivatievragen en voorkeursvragen (zie §3–§8 van de begeleidende tekst, Wieringa, 2026b). Het Leefstijlroer is binnen het project gekozen boven Positieve Gezondheid vanwege beheersbaarheid van onderhoud (Wieringa, 2026c).

4.2 React en Vite

React is een componentgebaseerde JavaScript-bibliotheek waarmee een interactieve UI declaratief opgebouwd kan worden uit herbruikbare bouwstenen. Vite is een moderne build-tool die gebruikmaakt van native ES-modules en biedt snelle hot module replacement (HMR), korte ontwikkelcycli en een lichte productie-build via Rollup (Refonte Learning, 2026). Voor een proof-of-concept dat lokaal draait, met een lokaal Ollama-LLM-proces aan de backendkant (Tran, 2026d), sluit deze combinatie aan op de scope-eis "het prototype draait lokaal" (Wieringa, 2026a).

Belangrijke aandachtspunten uit recente bronnen (2026):

- Strict Mode aanzetten tijdens ontwikkeling om bijwerkingen vroegtijdig op te sporen (rtCamp).
- ES-modules en code splitting standaard via Vite; geen aparte bundlerconfiguratie nodig.
- Geautomatiseerde toegankelijkheidstests via `vitest-axe/jest-axe` worden in 2026 als onderdeel van *definition of done* gezien (Code With Seb, 2026).

4.3 Toegankelijkheid en WCAG 2.2

Voor een tool die expliciet kwetsbare groepen bedient, laaggeletterden, ouderen, mensen met beperkingen, is **WCAG 2.2 niveau AA** het juiste richtsnoer (Code With Seb, 2026; AllAccessible). Concreet:

- Minimaal **4,5:1 contrast** voor normale tekst, **3:1** voor grotere tekst en UI-elementen.
- **Touch targets** van minimaal 44×44 px voor mobiel gebruik.
- Volledige **toetsenbordnavigatie** en logische focus-volgorde.
- **ARIA live regions** (`aria-live="polite"`) zodat schermlezers nieuwe chatberichten automatisch aankondigen (Reliasoftware, 2025).
- Tekstvergroting tot 200% zonder verlies van functionaliteit.
- **Semantisch HTML** als basis; ARIA-attributen alleen waar semantiek tekortschiet (React).

4.4 B1-taalniveau in chat én in microcopy

Taalniveau B1 betekent in de praktijk: korte zinnen (max. 15 woorden), actieve werkwoorden, geen jargon, alledaagse woorden en duidelijke instructies (Stichting Lezen en Schrijven). De actieve system prompt V2.1 verbiedt expliciet woorden als *socialiseren*, *frequentie*, *tijdsduur*, *connectie maken*, *faciliteren*, *optimaliseren* en schrijft *contact maken*, *hoe vaak*, *hoe lang*, *mogelijk maken* voor (Tran, 2026b). De frontend moet deze conventie overnemen in alle UI-teksten (knoplabels, foutmeldingen, placeholders), zodat de stijl binnen en buiten de chatbubbels consistent is.

4.5 Conversational UI

Op basis van recent UX-onderzoek zijn vier kwaliteitsaspecten leidend voor een chatinterface: *capability transparency*, *recovery patterns*, *confidence display* en *accessibility* (AI UX Design Guide; UXPin, 2026). Voor Padvinder vertaalt zich dat naar:

- Direct in de begroeting (§1 begeleidende tekst) duidelijk maken wat de bot wel en niet kan ("Ik help je drie activiteiten te vinden in Boxtel, Esch, Lennisheuvel en Liempde").
- Een **hybride invoer**: voornamelijk gestructureerde knoppen (radio buttons, sliders) met enkele open tekstvelden voor motivatievragen (Fuselab Creative, 2026).
- Een zichtbaar **terugknop/aanpassen-mechanisme** ("Klopt dit? Of wil je iets aanpassen?") zoals in §9 en §10 van de begeleidende tekst en in stap 4 van prompt V2.1 (Wieringa, 2026b; Tran, 2026b).
- Een **typing-indicator** of laad-indicator met *aria-live* voor transparantie over LLM-latentie.

5. Field-onderzoek

5.1 Doelgroep

De primaire doelgroep is breed (15–99 jaar) en bevat veel kwetsbare gebruikers met een lager taalniveau, mogelijke digitale ongeletterdheid en uiteenlopende fysieke beperkingen (Wieringa, 2026c). Dit heeft directe consequenties voor de frontend:

- Grote knoppen en ruime klikvlakken.
- Geen verborgen gestures of complexe interacties.
- Hoge leesbaarheid (font size ≥ 16 px, lijnhoogte ≥ 1.5).
- Mogelijkheid voor schermlezers en spraakbedieningsondersteuning.

5.2 Stakeholderkader

De opdracht komt van Carla Renders namens Renders Education (Wieringa, 2026c). Uit de probleemanalyse en de AVG-checklist blijkt dat zij waarde hecht aan:

- Het gesprek is zoveel mogelijk **anoniem**; geen onnodige persoonsgegevens (Wieringa, 2026a; Tran, 2026d).
- Een herkenbaar logo en visuele aansluiting bij de stichting (Wieringa, 2026a, *should have*).
- **Transparantie van matching**: de inwoner ziet welke voorkeuren en kenmerken hebben geleid tot een match (Tran, 2026d, §2.2 en §7).
- Een kort gesprek met drie instroompaden om de gesprekstijd en LLM-kosten te beperken.

5.3 Happy-flow-analyse

Uit het happy-flow-BPMN (Wieringa, 2026d) en de begeleidende tekst (Wieringa, 2026b) zijn vier herhalende UI-patronen af te leiden die de frontend moet ondersteunen:

1. **Slider-component** met emoji-uiteinden (: (... :)) per pijler en een score 0–100, conform ADR-001 (Tran, 2026f).
2. **Single-choice-vragen** (radio buttons) voor instroom, voorkeuren en afsluitende keuzes.
3. **Multi-choice-vragen** (checkboxen) voor lichamelijke beperkingen.
4. **Open tekstvelden** voor postcode (4-cijferig), leeftijd en motivatie (kort).

Daarnaast zijn er twee macropatronen: een **samenvattingsblok** (§9, "Klopt dit? Of wil je iets aanpassen?") en een **resultaatkaart** voor drie suggesties (§11).

5.4 Het datacontract uit het AI-team

Het AI-team heeft het datacontract grotendeels vastgelegd in `prompts.py`, ADR-001 en de AVG-checklist (Tran, 2026b; Tran, 2026d; Tran, 2026f). Voor de frontend zijn de volgende afspraken bindend:

- **Slider-meting**: 0 = ongelukkig, 100 = gelukkig. Hogere score = beter ervaren situatie. De frontend stuurt per pijler een object { "pijler": "bewegen", "score": 0..100 } (Tran, 2026f).
- **Aandachtspuntdrempel**: scores < 40 worden in de backend gemarkeerd als aandachtspunt en triggeren een motivatievraag; scores ≥ 70 mogen kort worden afgehandeld (Tran, 2026b; Tran, 2026f).
- **CONTEXT-blok**: het AI-team levert een kant-en-klaar CONTEXT-blok met de geselecteerde activiteiten. De frontend hoeft hier zelf geen string te bouwen, maar moet wel weten dat **alle voorgestelde activiteiten woordelijk uit dat blok komen**: verzonden extra details mag de UI dus niet visualiseren of suggereren (Tran, 2026b; Tran, 2026e).
- **Crisis-detectie**: gebeurt server-side (zie ook §8). De frontend hoeft geen eigen detectie te bouwen, maar moet een crisisroute-respons goed kunnen tonen.
- **Account en contact**: beide zijn opt-in. Account is **niet default**; contactgegevens worden pas na expliciete keuze gevraagd (Tran, 2026d, §7).

6. Workshop: ontwerp en prototyping

6.1 Voorgestelde componentstructuur

Op basis van de happy flow en het datacontract stel ik onderstaande componentstructuur voor, met scheiding tussen *presentational* en *container*-componenten:

- `<ChatWindow>`: root, beheert de scroll en focusmanagement.
- `<MessageList>`: render-loop, met `role="log"` en `aria-live="polite"`.
- `<BotMessage>` / `<UserMessage>`: chatbubbels met semantische HTML.
- `<SliderQuestion>`: slider 0–100, emoji-uiteinden, ARIA-attributen, toetsenbordbediening (pijltoetsen ± 5). Stuurt `{ pijler, score }` naar de state-reducer.
- `<ChoiceQuestion>`: radio- of checkboxgroep met grote tap targets.
- `<TextQuestion>`: `<input>` met `aria-label` en B1-placeholder; voor postcode beperkt tot 4 cijfers.
- `<SummaryCard>`: bewerkbare samenvatting van antwoorden (§9 happy flow) met "Klopt dit? Of wil je iets aanpassen?"-bevestiging.
- `<ActivityCard>`: resultaatkaart die de velden uit `activities.json` 1-op-1 toont (naam, pijlers, locatie_wijk, tijdstip, kosten, mobiliteit, individueel_of_groep). Toont géén verzonden extra velden.
- `<MatchReasoning>`: paneel onder de drie activiteiten dat per match aangeeft welke gebruikersvoorkeuren en pijlerscores tot de match hebben geleid. Verplicht door §7 van de AVG-checklist (Tran, 2026d).
- `<ConsentBanner>`: informeert over anonimiteit aan het begin (§1 happy flow).
- `<ResetButton>`: zichtbare "Gesprek opnieuw starten"-knop die de sessie-state wist (eis uit AVG-checklist §7 en §11).
- `<CrisisNotice>`: wordt getoond wanneer de server een crisis-respons stuurt; toont rustige doorverwijzing naar 113, huisarts of Veilig Thuis, zonder activiteiten te tonen.

6.2 State management

Voor een proof-of-concept met overzichtelijke state (één gesprek, één gebruiker per sessie) volstaat de combinatie van ``useState`` + ``useReducer`` + **Context API**. Externe state-bibliotheken (Redux, Zustand) zijn niet nodig en zouden de leercurve voor opvolgende projectgroepen onnodig verhogen, overdraagbaarheid is een DoD-eis (Wieringa, 2026a). De gespreks-state bevat minimaal:

```
{
  sessionId,                // UUID v4, alleen in-memory
  startedAt,
  entryType: "precies" | "beetje" | "nog_niet",
  scores: [
    { pijler: "bewegen", score: 0..100 },
    { pijler: "voeding", score: 0..100 },
    ...
  ],
  // shape conform ADR-001
  preferences: {
    binnenBuiten, alleenGroep,
    nederlandsSprekend, overdagAvond,
    budgetIsBelemmering, ...
  },
  limitations: string[],    // mobiliteit, allergieën, etc.
  additional: { gender?, leeftijd, postcode4 },
  consentContact: boolean, // opt-in
  consentAccount: boolean, // opt-in, default false
  matches: Activity[],     // exact zoals door backend teruggegeven
  matchReasoning: string[], // welke variabelen leidden tot de match
  crisisFlag: boolean      // gezet door backend-respons
}
```

Deze structuur sluit aan op (a) het BPMN (Wieringa, 2026d), (b) de scoreschema's van ADR-001 (Tran, 2026f) en (c) de minimum-dataverwerking uit de AVG-checklist (Tran, 2026d, §3 en §4).

6.3 Datacontract met de back-end

De frontend communiceert via een REST/JSON-laag met de FastAPI-back-end. De endpoints en responsvormen hieronder zijn **in afstemming met het backend-team** (Irsad en Nick): een deel is al geïmplementeerd, een deel moet nog gezamenlijk worden vastgelegd voordat de frontend er logica op bouwt. We beschrijven het contract daarom op eindpuntniveau (niet op bestandsniveau, want de backend-bestandsstructuur ligt nog niet vast) en sluiten af met een lijst open punten.

Gesprek

- **`POST /chat`**: body: { `sessionId`, `history`, `userInput`, `scores?` }. Response: { `botMessage`, `askedField?`, `crisis?` }. Crisis wordt server-side gedetecteerd (Tran, 2026e). Het veld `askedField?` is een **voorstel** waarmee de backend kan aangeven welk invoercomponent de frontend na een bot-bericht toont (slider, keuze of tekstveld). Dit veld zit nog niet in de backend-architectuur en wordt eerst samen gespecificeerd voordat de frontend er logica op bouwt.
- **Streaming** (een echte typing-indicator die het antwoord woord voor woord laat meelopen) vereist Server-Sent Events of WebSockets en is nog niet geïmplementeerd of afgesproken. Wacht de indicator alleen op de volledige respons, dan volstaat een gewone async **POST /chat**. Een eventueel **POST /chat/stream** leggen we vóór sprint 3 vast.

Een **crisis-response** heeft een vaste vorm zodat de frontend hem herkenbaar kan tonen: { "crisis": true, "crisisMessage": "...", "referrals": [{ "label": "113 Zelfmoordpreventie", "value": "0800-0113" }, ...] }. Staat **crisis** op **true**, dan toont de frontend `<CrisisNotice>` met de doorverwijzingen en géén activiteiten.

Activiteiten en matching

In de huidige backend-architectuur zit de matching **ingebakken in de gespreksbeurt**; er is (nog) geen los `match`-endpoint. We moeten daarom afspreken of de matches en de onderbouwing terugkomen via de laatste `POST /chat`-respons, dan wel via een apart `GET /activities`-endpoint met de filters als query-parameters. Hoe we het ook afspreken, een **Activity** geeft **uitsluitend de velden uit `activities.json`** terug: `id`, `naam`, `pijlers[]`, `locatie_wijk`, `tijdstip`, `kosten`, `mobiliteit`, `individueel_of_groep`, `beschrijving`. De frontend rendert geen veld dat niet in deze lijst staat (Tran, 2026a; Tran, 2026b).

Account en contact (opt-in)

- `POST /users`: maakt alleen na expliciete opt-in een account aan.
- `DELETE /users/{id}`: accountverwijdering; verplicht aanwezig in PoC volgens AVG-checklist §11.
- `POST /users/{id}/contact-requests`: alleen na expliciete keuze, met minimale velden (naam + e-mail óf naam + telefoonnummer).

Validatie aan de frontendkant. De 4-cijferige postcode loopt aan de backendkant als harde filter buiten het LLM om (via ChromaDB) en wordt daar nog niet op geldigheid gecontroleerd. De frontend valideert de postcode dus zelf (precies 4 cijfers) voordat hij wordt meegestuurd.

Open punten (af te stemmen met de backend)

- **Matching-locatie**: komen de matches terug via de `POST /chat`-respons of via een apart `GET /activities`-endpoint?
- **Vorm van `reasoning`**: levert de backend de match-uitleg als gestructureerde data (bijv. per match een set { `pijler`, `voorkeur`, `belemmering` }) of als vrije tekst? Dit staat nog nergens vastgelegd; wij stellen voor hier een mini-ADR van te maken voordat de frontend `<MatchReasoning>` definitief bouwt.
- `askedField`: wel of niet opnemen, en zo ja met welke waarden.
- **Streaming**: SSE/WebSockets of een gewone async POST (besluit vóór sprint 3).

Door het contract klein te houden, de velden en de crisis-response vooraf vast te leggen én bovenstaande open punten expliciet te beslissen, blijft de frontend losgekoppeld van de LLM-keuze en kunnen back-end- en frontend-teams parallel werken.

6.4 Styling en visuele identiteit

Voorstel: **Tailwind CSS** of CSS-modules voor scoped styling, met een design-token-systeem voor kleuren en typografie. Het logo van de stichting wordt opgenomen in de header (Wieringa, 2026a, *should have*). Kleurcontrast wordt vooraf gevalideerd met een contrast-checker.

6.5 Routing en build

Voor een single-page chatbot is **React Router** alleen nodig voor de `/account`-route (§12 begeleidende tekst). Anders volstaat één hoofdweergave. Vite verzorgt de build met code splitting per route.

6.6 Mockup: match-uitleg (deelvraag 7)

Deelvraag 7 vraagt hoe de frontend zichtbaar maakt *waarom* een activiteit een match is. Onderstaande mockup maakt dat concreet: hij laat zien wat de inwoner per suggestie te zien krijgt, de activiteit (`<ActivityCard>`) met daaronder de onderbouwing (`<MatchReasoning>`). De onderbouwing benoemt expliciet drie dingen: welke **pijler**, welke **voorkeur** en welke **belemmering** is meegewogen. Zo is meteen duidelijk welke velden het AI-team in het JSON-antwoord (`reasoning`) moet teruggeven.

```
+-----+
| Wandelclub De Dommel          (buiten) |
| Pijler: Bewegen | Liempde | gratis      |
| Elke woensdag 10:00 | groep | rollator-ok |
+-----+
| Waarom past dit bij jou?         |
| - Pijler: 'bewegen' lage score (30/100) |
|   => aandachtspunt                |
| - Voorkeur: je wilt graag samen + buiten |
| - Belemmering: rollator => locatie is   |
|   rollator-toegankelijk            |
+-----+
```

Per match koppelt `<MatchReasoning>` dus één pijlerscore, één of meer voorkeuren en (indien van toepassing) één belemmering aan de activiteit. Dat maakt de matching transparant (eis van Carla en AVG-checklist §7, Tran, 2026d) en bepaalt tegelijk welke velden de backend in `reasoning` levert: de gematchte pijler met score, de meegewogen voorkeuren en de meegewogen belemmering.

7. Lab: test- en validatieplan

Conform de "validated solution"-aanpak van het DOT Framework (HBO-i) worden ontwerpkeuzes pas geaccepteerd na toetsing. Het is daarbij belangrijk niet alleen vast te leggen *wat* we testen, maar ook *wanneer*. Wij hanteren vier momenten: **continu in CI** bij elke commit/PR (de geautomatiseerde tests), **per sprint** tijdens het bouwen van een component, **vóór de interne demo** en **vóór de eindoplevering**. Hieronder staat per test wat er getoetst wordt en op welk moment.

1. **Unit tests** met Vitest voor componenten (slider-logica conform ADR-001, state-reducer, score-aandachtspuntmapping). *Wanneer*: continu in CI, en bij het bouwen van elk nieuw component (test-first waar mogelijk).
2. **Geautomatiseerde toegankelijkheidstests** met `vitest-axe` op alle interactieve componenten (Code With Seb, 2026). *Wanneer*: continu in CI; een falende a11y-test blokkeert de merge.
3. **Handmatige toetsenbordnavigatietest** door de volledige happy flow (Tab, Shift+Tab, Enter, Space, pijltjestoetsen op slider). *Wanneer*: per sprint zodra een flow-stap af is, en als regressiecheck vóór de demo.
4. **ScherMLEZERTEST** (NVDA en VoiceOver) op de chatlog, sliders en `<MatchReasoning>`. *Wanneer*: vóór de interne demo en opnieuw vóór de eindoplevering.
5. **Kleurcontrasttest** met de WCAG Contrast Checker. *Wanneer*: eenmalig bij het vastleggen van de design-tokens en daarna bij elke kleurwijziging.
6. **Contract-test**: de frontend stuurt `{pijler, score}`-objecten exact volgens ADR-001 en accepteert alleen `<ActivityCard>`-velden die het backend-team in `activities.json` documenteert. Geen frontend-veld dat niet in de dataset bestaat. *Wanneer*: continu in CI, en direct na elke wijziging in het API-contract met de backend.
7. **Privacy-test**: prompt-injectie via vrije tekstvelden, vraagt de UI wel correct geen medische of identificerende gegevens uit (Tran, 2026d, §4)? Wordt input van de gebruiker niet ergens in `localStorage` of `sessionStorage` opgeslagen zonder opt-in? *Wanneer*: bij elke PR die invoervelden of opslag raakt, en als eindcheck vóór oplevering.
8. **Mini-usability-test** met 3 medestudenten en, indien mogelijk, 1 representatieve eindgebruiker uit Boxtel. *Wanneer*: na de eerste werkende happy-flow-iteratie, vóór de demo aan Carla.
9. **Performance-meting** met Lighthouse (doel: Performance ≥ 90 , Accessibility ≥ 95 op een lokale build). *Wanneer*: vóór de interne demo en vóór de eindoplevering, op een productie-build.

8. Showroom: expertreview en referentiestudie

Naast Lab-validatie wordt de Showroom-strategie ingezet om de ontwerpkeuzes door vakgenoten te laten beoordelen (HBO-i).

1. **Expertreview met Fontys-docenten/experts** op minimaal drie onderwerpen: (a) componentstructuur en React/Vite-architectuur, (b) toegankelijkheid (WCAG 2.2 AA, ARIA-gebruik, sliders), en (c) conversational UX, B1-microcopy en transparantie van de matchingvariabelen.
2. **Peer review binnen de projectgroep**: wireframes en componentcontracten worden vóór implementatie samen met het AI-team en de Product Owner gecontroleerd op contractconsistentie met `prompts.py` en de AVG-checklist.
3. **Referentiestudie**: vergelijking met bestaande, publiek toegankelijke chatinterfaces (overheids- en gemeentelijke assistenten) om patronen voor begroeting, instroomkeuze, samenvatting en resultaatkaarten te benchmarken.
4. **Demo-sessie** met Carla Renders na de eerste werkende iteratie, met als focus inhoudelijke aansluiting bij Renders Education en de transparantie-eisen.

De Showroom-uitkomsten worden gebruikt als input voor de volgende Workshop-iteratie.

9. Privacy by design in de frontend

Privacy is uitdrukkelijk een must-have (Wieringa, 2026a). De AVG-checklist op PoC-niveau (Tran, 2026d) maakt expliciet welke principes de frontend moet borgen:

- **Doelbinding en dataminimalisatie**: de frontend vraagt geen BSN, geen geboortedatum (alleen leeftijd in jaren), geen volledig adres (alleen 4-cijferige postcode), geen medische diagnoses, geen inkomensgegevens (Tran, 2026d, §4).
- **Anonieme `sessionId`** (UUID v4) als enige identifier tijdens het gesprek; alleen in-memory.
- **Geen tracking-scripts** of third-party-analytics in het PoC.
- **Geen persoonsgegevens** in `localStorage` of `sessionStorage` zonder expliciete opt-in.
- **Duidelijke microcopy**: "Dit gesprek is anoniem" zichtbaar in de begroeting (§1 happy flow).
- **Reset-knop**: zichtbare "gesprek opnieuw starten"-knop die de hele sessie-state wist (Tran, 2026d, §11).
- **Transparantie van matching**: `<MatchReasoning>` toont voor elke aanbevolen activiteit welke pijlerscores en voorkeuren tot de match hebben geleid. Dit is een eis van Carla én van de AVG-checklist (Tran, 2026d, §2.2 en §7).
- **Spontaan gedeelde gevoelige info wordt niet herhaald**: de system prompt borgt dit aan LLM-zijde; de frontend toont nooit verzonden of teruggekoppelde medische details (Tran, 2026d, §11; Tran, 2026b).
- **Account is opt-in en standaard uit** (Tran, 2026d, §11).

10. Risico's en aandachtspunten

Risico	Impact	Mitigatie
LLM-latentie (lokale Ollama) verstoort gespreksritme	Hoog	Typing-indicator + skeleton + max-timeout met fallback-bericht.
Slider niet bedienbaar via toetsenbord	Hoog (toegankelijkheid)	<code><input type="range"></code> met aangepaste styling; pijltjestoetsen ± 5 .
Frontend toont velden die niet in <code>activities.json</code> bestaan	Middel	Contract-test: alleen velden uit de dataset renderen; type-checked schema.
Ongeldige postcode bereikt de ChromaDB-filter (backend vangt dit niet af)	Middel	Frontend valideert de postcode (precies 4 cijfers) vóór verzending, met een duidelijke foutmelding op B1.
Inwoner deelt spontaan medische info via vrij tekstveld	Middel	UI-melding "Je hoeft geen medische gegevens te delen" bij open velden; backend zorgt voor LLM-gedrag.
Frontend slaat gevoelige data op in <code>localStorage</code> zonder opt-in	Hoog (privacy)	State strikt in-memory; reviews op elke PR die storage raakt.
Crisis-respons niet duidelijk getoond	Hoog	Dedicated <code><CrisisNotice></code> -component; geen activiteitenkaarten tonen wanneer <code>crisisFlag</code> true is.
Doelgroep begrijpt instroomkeuze niet	Middel	A/B testen van bewoording, B1-validatie.
Overdracht naar volgende projectgroep mislukt	Middel	Heldere README, gedocumenteerde componenten, JSDoc-commentaar.

11. Conclusie

De frontend kan onderbouwd gebouwd worden met **React + Vite**, een **componentstructuur die het Leefstijlroer 1-op-1 ondersteunt** (slider, choice, summary, activity card, match-reasoning), een **lichte state-managementaanpak** met `useReducer` + Context, een **strikte WCAG 2.2 AA + B1-discipline** en een **strikte aansluiting op het datacontract van het AI-team** (ADR-001-sliderscore, geen verzonnen velden, opt-in account/contact). Het happy-flow-BPMN, de begeleidende tekst en `prompts.py` leveren samen een duidelijk script dat zich goed laat vertalen naar herbruikbare componenten. De grootste aandachtspunten zijn toegankelijkheid (sliders en live regions), B1-microcopy, transparantie van matching (eis Carla én AVG-checklist) en privacy by design (anonimiteit tot §12). De analyse vormt de basis voor het adviesdocument, waarin de concrete aanbevelingen voor Carla Renders worden samengevat.

Bronnen

AI UX Design Guide. *Conversational UI, Design Patterns for Chat Interfaces & Conversational UX*.
<https://www.aiuxdesign.guide/patterns/conversational-ui>

AllAccessible. *React Accessibility: Best Practices Guide for WCAG-Compliant SPAs*.
<https://www.allaccessible.org/blog/react-accessibility-best-practices-guide>

Autoriteit Persoonsgegevens. (2024). *Algemene verordening gegevensbescherming*.
<https://www.autoriteitpersoonsgegevens.nl/>

Sahel Nawabi. (2026a). *Analysedocument Frontend AI-chatbot Sociaal Knooppunt* (Versie 1.0) [Intern projectdocument]. Projectgroep Sociaal Knooppunt.

Code With Seb. (2026). *Web Accessibility in 2026: The Frontend Developer's Survival Guide*.
<https://www.codewithseb.com/blog/web-accessibility-2026-eaa-ada-wcag-guide>

Fuselab Creative. (2026). *Chatbot UI Design Patterns and Best Practices 2026*.
<https://fuselabcreative.com/chatbot-interface-design-guide/>

HBO-i. *The DOT Framework*. ICT Research Methods. <https://ictresearchmethods.nl/dot-framework/>

HBO-i. *ICT Research Methods*. <https://ictresearchmethods.nl/>

React. *Accessibility*. React Documentation. <https://legacy.reactjs.org/docs/accessibility.html>

Refonte Learning. (2026). *Front-End Development and Vite in 2026: Top Trends, Tools, and Skills for the Modern Web*. <https://www.refontelearning.com/blog/front-end-development-and-vite-in-2026-top-trends-tools-and-skills-for-the-modern-web>

Reliasoftware. (2025). *Build A Production-Ready React AI Chatbot with Streaming, RAG, & More*.
<https://reliasoftware.com/blog/react-production-ready-ai-chatbot>

rtCamp. *React Accessibility (A11y) Best Practices and Guidelines*.
<https://rtcamp.com/handbook/react-best-practices/accessibility/>

Stichting Lezen en Schrijven. *Schrijven op taalniveau B1*. <https://www.lezenenschrijven.nl/>

Renders Education. *Leergebied@ Gezond*. <https://www.richter.education>

Tran, T. (2026a). *activities.json: synthetische dataset Sociaal Knooppunt* (Sprint 1) [Intern projectdocument]. Projectgroep Sociaal Knooppunt.